Capacity-Aware SDN Orchestration of V2X Network Slices with Metering, Strict Priority and Mobility Handling on an Emulated Vehicular Edge Testbed

A. Adharsh, Rishabh Bhat, Samuel Gerald and Navin Kumar
Department of Electronics and Communication Engineering, Amrita School of Engineering, Bengaluru
Amrita Vishwa Vidyapeetham, India
Email: {bl.en.u4ece23101, bl.en.u4ece23240, bl.en.u4ece23245}@bl.students.amrita.edu

***Abstract*—Vehicles share roadside infrastructure among traffic classes with very different needs, and safety messages suffer once bulk infotainment saturates the backhaul. This paper presents an orchestrator, written as a RYU application over OpenFlow 1.3, that maps vehicle-to-everything (V2X) traffic onto five slices, polices each slice with a switch meter, tags packets with DSCP for strict-priority backhaul queues, tracks vehicles across roadside-unit handovers, and every 2 s resizes the broadband slice to the capacity left by critical traffic. We evaluate it on a Mininet-WiFi testbed with six vehicles, two roadside units and edge servers, using an ablation of five configurations over four scenarios with five repetitions (80 trials). Under a broadband flood, the 95th-percentile platoon delay falls from 3.8 s to 24.9 ms with no loss. Under heavy critical load, metering alone gives 203 ms, whereas the orchestrator holds 23.2 ms. The open, scripted testbed lets researchers and operators prototype and compare vehicular slicing policies on a single machine.**

***Index Terms*—V2X, network slicing, software-defined networking, OpenFlow meters, strict-priority scheduling, Mininet-WiFi, edge computing**

I. Introduction

Connected and automated driving depends on a steady exchange of messages between vehicles, roadside units (RSUs) and servers. These services ask very different things of the network. Cooperative awareness and emergency warnings are small and periodic, yet they must arrive within tens of milliseconds with high reliability; platooning and sensor sharing tighten these bounds further [1]. In the same cell, passengers stream video and download maps, which demands throughput but tolerates delay. The New Radio sidelink and its cellular counterpart give vehicles the radio means to carry both kinds of traffic [2]. However, the RSU uplinks and the aggregation links behind them are shared and of finite capacity, so traffic of one kind can still delay traffic of another.

Network slicing answers this by carving one physical infrastructure into logical networks, each with its own resources and service guarantees [3]. Its practical form depends on softwarization: the control logic is moved out of individual devices into programmable functions that can be reconfigured at run time [4]. Software-defined networking (SDN) supplies the needed mechanism. With OpenFlow, a logically centralized controller installs match–action rules in switches, reads their counters, and changes their behaviour without touching hardware [5], and the approach has matured into a broad ecosystem of controllers and data planes [6]. Applied to vehicular networks, slicing lets an operator give safety, cooperative-driving, infotainment and telemetry traffic separate treatment over the same roadside equipment [7].

Several studies have brought these ideas into vehicular settings. Early software-defined VANET designs showed that a central controller can steer vehicular flows and adapt routes [8]. Campolo *et al.* used OpenFlow metering over an emulated wireless network to keep automotive slices apart [9], and Saraiva *et al.* built an application-driven framework that combines a RYU controller, an emulated access network and a road simulator, and compared it with best-effort and QoS-only operation [10]. These works rely on emulators such as Mininet-WiFi, which runs real network stacks with a wireless channel model on one host [11]. Three gaps remain. First, slice budgets are usually fixed, or are changed on coarse triggers that ignore how much capacity the shared link actually has left. Second, rate policing and packet scheduling are rarely evaluated together, so it is unclear when policing alone is enough. Third, practical pitfalls of Wi-Fi roadside units, such as stale station entries after a handover and traffic bridged inside the access point, are seldom reported, although they silently break slice isolation.

This paper addresses these gaps. Its original contributions and novelty are:

- **A closed-loop, capacity-aware reallocation policy** that reads OpenFlow meter counters every 2 s and sets the broadband slice to the capacity the critical slices leave on the backhaul, with step quantization, a floor, and explicit protect/adjust/release states.
- **Joint enforcement by meters and strict priority**: per-slice meters for isolation, plus controller-written DSCP marks that drive a three-band strict-priority queue on both ends of every backhaul link, without disturbing the emulated link rate and delay.
- **Mobility-aware forwarding for Wi-Fi RSUs** that detects

handovers from port changes and flushes stale rules network-wide, and two fixes that keep slicing intact across mobility: fast station expiry with vehicle beacons, and client isolation with hairpin forwarding so that same-RSU traffic cannot bypass its meter.
- **A reproducible ablation benchmark**: five configurations that add one mechanism at a time, four scenarios including scripted handover and random-waypoint mobility, five repetitions each (80 trials), with per-packet latency, jitter, loss and outage metrics, all released with the testbed scripts and a replay dashboard.

The remainder of the paper is organized as follows. Section II describes the system architecture. Section III details the design of the slices, the enforcement mechanisms, the reallocation loop, the mobility handling and the measurement method. Section IV presents and discusses the results, and Section V concludes with the challenges faced and directions for future work.

II. System Architecture

Fig. 1 shows the testbed. The data plane consists of three Open vSwitch instances controlled over OpenFlow 1.3. Two of them, *rsu1* and *rsu2*, are IEEE 802.11g access points on channels 1 and 6 with a 30 m range, and each has a multi-access edge computing (MEC) server attached by a 0.5 ms link. The third, *s1*, aggregates the two RSUs over backhaul links of 20 Mb/s and 5 ms each, and connects a central infotainment server and a telemetry server over a 100 Mb/s, 15 ms link. The backhaul is deliberately the bottleneck of the network. Six vehicles are Wi-Fi stations; they either follow scripted, intersecting paths that force handovers between the RSUs, or move under a seeded random-waypoint model at 2–4 m/s. The whole network runs in Mininet-WiFi with the wmediumd channel model [11], so association, signal loss and re-association are emulated rather than assumed.

The control plane is a RYU 4.34 controller hosting the orchestrator, which is organized in five modules (Fig. 1). The *slice registry* holds each slice's traffic signature, meter identifier, budget and DSCP value. The *rule and meter installer* programs every switch as soon as it connects. The *meter monitor* polls meter statistics and runs the reallocation loop. The *mobility manager* learns where each vehicle is and reacts to handovers. A northbound *REST interface* on port 8081 lists slices and events and lets an operator resize any slice at run time.

Fig. 2 traces a packet through a switch. Table 0 classifies it by transport protocol and port, in both directions so that replies are classified too; the matching rule sends it through the slice meter, writes the slice's DSCP code point, and passes it to table 1, which forwards it on the learned port. On a backhaul port, the packet then enters a Linux queueing tree: a hierarchical token bucket (HTB) shaper at the link rate, a three-band strict-priority scheduler that reads the DSCP field, and a 5 ms delay stage per band. Placing priority under the shaper keeps the emulated link rate and delay unchanged while deciding which packet leaves first.

TABLE I
Slice configuration (budgets apply on every switch)

Slice	V2X service	Match	Meter, b_s	DSCP	Band
URLLC	safety alerts to edge	UDP 5000	1, 5 Mb/s	46	1
Platoon	cooperative driving	UDP 9000	2, 4 Mb/s	40	1
eMBB	infotainment	TCP 8080	3, 12 Mb/s*	10	3
mMTC	telemetry (MQTT)	UDP 1883	4, 1 Mb/s	18	2
Best effort	all other traffic	–	5, 2 Mb/s	0	3

*Adjusted at run time by the reallocation loop (Section III-D).

III. Design and Development

A. Slice Model

Table I lists the slices. Let $\mathcal{S}$ be the slice set, b_s the budget of slice s in kb/s, and $C = 20$ Mb/s the backhaul capacity. The budgets are deliberately oversubscribed,

$$\sum_{s\in\mathcal{S}} b_s = 24 \text{ Mb/s} > C, \tag{1}$$

as in any statistically multiplexed network: each slice may burst to its budget, but not all at once. The critical set is $\mathcal{K} = \{\text{URLLC}, \text{platoon}\}$.

B. Policing with OpenFlow Meters

Each slice rule points to a meter with a single drop band, which behaves as a token bucket. Tokens accumulate at the slice rate b_s up to a depth β_s; a packet of L bits is forwarded only if enough tokens remain:

$$\begin{aligned} T_s(t) &= \min\big(\beta_s,\ T_s(t_0) + b_s\,(t - t_0)\big), \\ \text{forward} &\iff T_s(t) \geq L, \quad T_s \leftarrow T_s - L. \end{aligned} \tag{2}$$

We set $\beta_s = b_s \times 0.1$ s, which allows short bursts but bounds the long-run rate by b_s. Meters enforce isolation: no slice can take more than its budget, whatever the others do.

C. Marking and Strict-Priority Scheduling

Policing alone does not decide who waits when the link is full, and (1) allows that to happen. The orchestrator therefore writes a Differentiated Services code point [12] into every classified packet (Table I), and the backhaul scheduler always serves band 1 before band 2 and band 2 before band 3. Since the scheduler is non-preemptive, a critical packet waits at most for the low-priority packet already on the wire, plus the critical backlog ahead of it:

$$W_1 \leq \frac{L_{\max}}{C} + \frac{Q_1}{C}, \qquad \frac{L_{\max}}{C} = \frac{1514 \times 8}{20 \times 10^6} \approx 0.6 \text{ ms}, \tag{3}$$

where Q_1 is the band-1 backlog. With a single first-in first-out queue, the wait is instead Q/C for the *total* backlog Q, which a TCP flow keeps large.

D. Capacity-Aware Reallocation Loop

Every $\Delta t = 2$ s, the monitor requests meter statistics from each switch and estimates each slice's offered rate from the byte counter B_s:

$$\hat{r}_s(k) = \frac{8\,\big(B_s(k) - B_s(k-1)\big)}{t_k - t_{k-1}}. \tag{4}$$

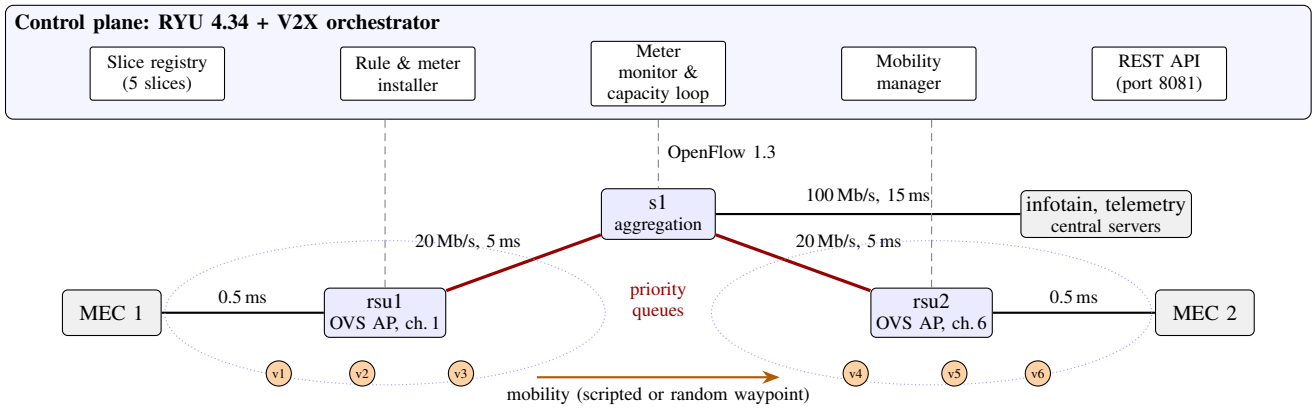


Fig. 1. Testbed architecture. The orchestrator runs as a RYU application and controls both RSUs and the aggregation switch *s1*; strict-priority queues sit on both ends of each 20 Mb/s backhaul link.

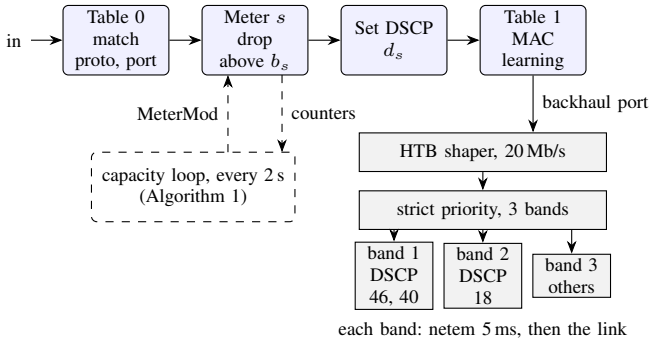


Fig. 2. Per-switch pipeline: classification, metering and marking, MAC forwarding, and the strict-priority queue on each backhaul port. Dashed arrows show the reallocation loop.

The critical load is $R_c(k) = \sum_{s\in\mathcal{K}} \hat{r}_s(k)$, and the broadband target is the spare capacity after a headroom $H = 1.5$ Mb/s for mMTC, best effort and bursts, quantized to steps of $\delta = 0.5$ Mb/s and clipped between a floor $e_{\min} = 2$ Mb/s and the nominal budget:

$$e(k) = \min\Big(b_{\text{eMBB}},\ \max\big(e_{\min},\ \delta\lfloor (C - R_c(k) - H)/\delta \rfloor\big)\Big). \tag{5}$$

Protection is active whenever $e(k) < b_{\text{eMBB}}$, that is, whenever $R_c > C - H - b_{\text{eMBB}} = 6.5$ Mb/s. Algorithm 1 shows the per-switch policy. The step δ prevents the meter from being rewritten on every small fluctuation, and the floor keeps infotainment alive under any load.

E. Mobility-Aware Forwarding

Table 1 performs MAC learning with 10 s idle and 30 s hard time-outs. Algorithm 2 summarizes the packet-in handler. A handover is detected when a known source address arrives on a new port; the vehicle's forwarding rules are then deleted on every switch, so the next packets are learned along the new path. A destination that appears to lie behind the wired port a packet entered on is stale and is rediscovered by flooding.

Two Wi-Fi effects had to be handled for slicing to survive mobility. First, an RSU kept a departed vehicle in its station table and delivered frames to it locally; only 74 of 90 vehicle pairs could reach each other after the vehicles moved. Following the configuration published with [10], the access points now expire silent stations after 6 s, and every vehicle sends a 1 s beacon, which restored 90/90 connectivity. Second, two vehicles on the same RSU were bridged inside the access point, so their packets never reached OpenFlow and escaped their meter. Client isolation now forces this traffic into the switch, and the controller returns it on the ingress Wi-Fi port (`OFPP_IN_PORT`), so it is classified, metered and counted like any other flow.

Algorithm 1 Capacity-aware reallocation (per switch, every 2 s)

read byte counters B_s; compute $\hat{r}_s$ by (4); $R_c \leftarrow \sum_{s\in\mathcal{K}} \hat{r}_s$
compute e by (5)
if $e < b_{\text{eMBB}}$ **then**
  if not protecting **then**
    protecting $\leftarrow$ true; $e^* \leftarrow e$; log PROTECT_ON
  else if $|e - e^*| \geq \delta$ **then**
    $e^* \leftarrow e$; log ADJUST
  end if
else if protecting **then**
  protecting $\leftarrow$ false; $e^* \leftarrow b_{\text{eMBB}}$; log PROTECT_OFF
end if
if installed eMBB rate $\neq e^*$ **then** send METERMOD(e^*)

Algorithm 2 Mobility-aware packet-in handling

Require: packet with source a, destination z, ingress port p on switch d
if a known on switch d at port $p' \neq p$ **then**
  log HANDOVER; delete all rules for a on every switch
end if
learn $a \mapsto p$; $q \leftarrow$ learned port of z (else FLOOD)
if $q = p$ and p is wired **then**
  log STALE; delete rules for z; $q \leftarrow$ FLOOD
end if
if $q = p$ and p is Wi-Fi **then**
  $q \leftarrow$ IN_PORT ▷ hairpin, same RSU
end if
install rule $(p, a, z) \rightarrow q$ unless flooding; forward packet

TABLE II
ABLATION CONFIGURATIONS

Configuration	Meters	DSCP priority	Reallocation loop
Best effort (BE)	–	–	–
Meters only (MO)	✓	–	–
Meters + loop (ML)	✓	–	✓
Meters + priority (MP)	✓	✓	–
Orchestrated (OR)	✓	✓	✓

TABLE III
FUNCTIONAL VALIDATION OF THE ENFORCEMENT MECHANISMS

Test	Offered	Delivered	Limit
URLLC UDP flood	10 Mb/s	4.96 Mb/s	5 Mb/s
mMTC UDP flood	2 Mb/s	0.99 Mb/s	1 Mb/s
Best-effort UDP flood	4 Mb/s	1.97 Mb/s	2 Mb/s
eMBB TCP download	unlimited	11.27 Mb/s	12 Mb/s
URLLC after REST resize	10 Mb/s	2.98 Mb/s	3 Mb/s
Connectivity after mobility (pairs)	74/90 before fixes, 90/90 after		

F. Traffic and Measurement

Critical and telemetry traffic is generated by a purpose-built UDP probe: the sender stamps each numbered packet, the receiver echoes it, and the sender records the round-trip time (RTT) of every packet or marks it lost. Four probe flows run in every trial: URLLC from a vehicle to its own RSU's MEC (*edge*), URLLC to the other RSU's MEC across the backhaul (*far*), platoon traffic between vehicles on different RSUs, and mMTC to the telemetry server. Light flows use 100 packets/s; in the heavy scenario the far URLLC and platoon flows are raised to 4.3 and 3.5 Mb/s. Congestion comes from a 30 s iperf3 download of four parallel TCP streams from the central server. For a window with N sent packets, M of them delivered with RTTs $x_1, \ldots, x_M$ at receive times $\tau_1, \ldots, \tau_M$, we report

$$\text{loss} = 1 - \frac{M}{N}, \qquad G = \max_i (\tau_{i+1} - \tau_i), \tag{6}$$
$$J = \frac{1}{M-1} \sum_{i=2}^{M} |x_i - x_{i-1}|,$$

together with the mean and the 95th and 99th percentiles of x. Here J is the jitter and G the longest delivery gap a vehicle would notice. Windows cover the flood (*load*), the period before it (*idle*), or the vehicles' movement (*moving*).

G. Experimental Design

Table II defines the ablation: each configuration adds one mechanism, so that its contribution can be isolated. Four scenarios are run: *congestion* (parked vehicles, light critical traffic, then the flood), *heavy* (critical load raised so that (1) bites), *handover* (scripted movement across RSUs) and *random* (random-waypoint mobility with the flood). Every trial starts from a clean emulator and controller and lasts about two minutes. The parked scenarios use all five configurations and the mobile ones use three, each repeated five times, giving 80 trials. Results are reported as the mean and standard deviation over repetitions.

IV. RESULTS AND DISCUSSION

A. Functional Validation

Table III confirms that every mechanism behaves as designed before any comparison is made. The meters cap each slice within 1.5% of its budget, and a REST request that lowered URLLC to 3 Mb/s took effect immediately. The priority tree is exercised as intended: in a heavy-load trial, about 61 MB crossed band 1 of the backhaul when marking was enabled and none when it was disabled. With the network idle, a vehicle reaches its own RSU's MEC in 1.2 ms, against 21 ms for the other RSU's MEC and 40.7 ms for the central servers, which supports placing safety services at the edge.

B. Congestion

Fig. 3(a) shows the flood scenario. Without slicing, the four TCP streams fill the backhaul queue and every other flow waits behind them, the effect known as bufferbloat [13]: the 95th-percentile platoon RTT reaches 3.79 s and URLLC to the far MEC 1.63 s, while platoon and mMTC lose 7.7% and 9.8% of their packets. Every slicing configuration removes the problem: eMBB is held to its budget (16.9 Mb/s unrestricted against 11.3 Mb/s, Fig. 4), the link no longer overflows, and every probe flow stays within 4.5 ms of its idle 95th-percentile RTT with zero loss. Here the loop never activates, as (5) predicts for $R_c \approx 1.1$ Mb/s, and priority adds a small gain only (platoon 24.9 ms instead of 25.5 ms, jitter 1.5 ms instead of 1.7 ms). In this regime, isolation alone is sufficient.

C. Heavy Critical Load

Fig. 3(b) is the decisive case. With 7.9–8.6 Mb/s of critical traffic, eMBB at its full budget pushes the backhaul past capacity, as (1) permits. Metering alone then leaves critical packets queued behind admitted eMBB packets: the platoon RTT is 203 ms and URLLC-far 203 ms, a 3.5-fold improvement over best effort (702 ms) but far from the idle 21 ms. Each of the two remaining mechanisms closes the gap on its own, by a different route. The loop fired PROTECT_ON on all three switches in every heavy trial, lowered eMBB from 12 to 10.5 and then 9.5 Mb/s as critical load grew, and released it when the flows ended; the link no longer overflows, and the platoon RTT drops to 24.4 ms at the cost of 2.3 Mb/s of eMBB throughput (Fig. 4). Strict priority instead lets critical packets overtake the backlog, as (3) suggests, giving 23.3 ms while eMBB keeps 11.2 Mb/s. The full orchestrator achieves 23.2 ms for platoon and 22.1 ms for URLLC-far, the lowest of all configurations. The edge URLLC flow stays at 1.8 ms in every case, because it never crosses the backhaul.

The two mechanisms are complementary rather than redundant. Priority is free for eMBB but only protects links whose queues the operator controls. The loop works from measurements alone and prevents the overload network-wide, which matters where a queue cannot be configured, at the

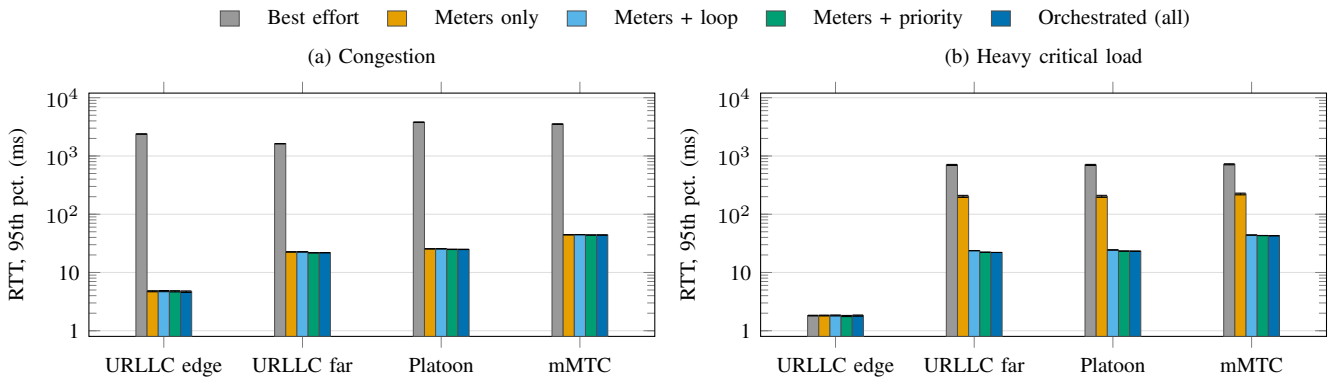


Fig. 3. 95th-percentile RTT during the eMBB flood (log scale; mean of five repetitions, whiskers show one standard deviation). (a) Light critical load. (b) Heavy critical load, where the slice budgets exceed the backhaul capacity.

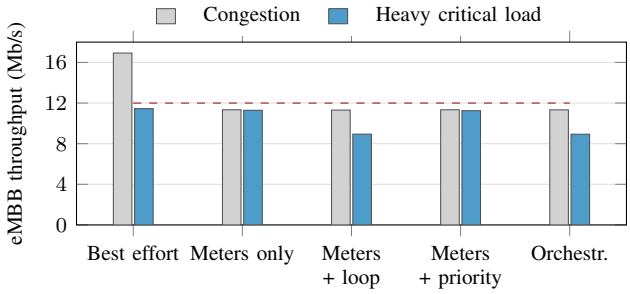


Fig. 4. eMBB throughput during the flood. The dashed line marks the nominal 12 Mb/s budget; the loop lowers it only under heavy critical load.

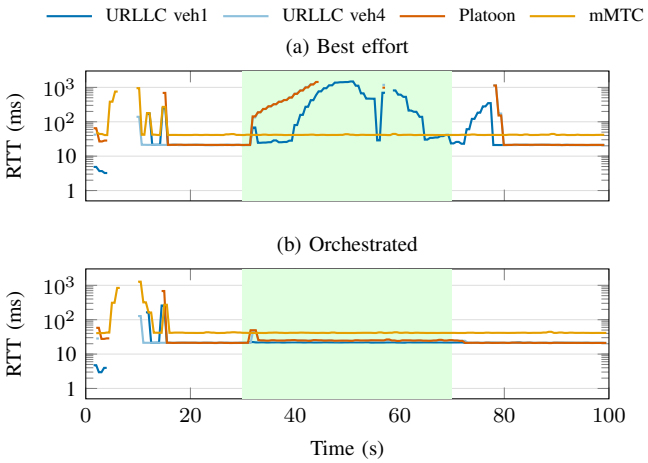


Fig. 5. Per-second 95th-percentile RTT in one scripted run. The shaded band marks the eMBB flood.

price of a temporary reduction for the broadband slice. Across repetitions, the standard deviation of the 95th-percentile RTT stays within 4% of its mean for every flow and configuration of the parked scenarios, so the differences are not artefacts of a single run.

D. Temporal Behaviour

Fig. 5 follows one scripted run second by second, with the same vehicles, paths and flood in both configurations. The spikes in the first 16 s coincide with the handovers and are alike in both runs. During the flood (30–70 s), best effort drives platoon and URLLC towards 1 s, whereas the orchestrated run stays flat near 22 ms, and the mMTC flow to the central server stays near 41 ms.

E. Mobility

Fig. 6 reports packet loss while the vehicles move. Slicing roughly halves the loss of the moving flows (platoon 81.8% to 50.5%, edge URLLC 60.6% to 26.7%), because their packets no longer queue behind the concurrent download. Under random waypoint the trend is the same (platoon 47.4% to 19.4–21.8%), but the spread between seeds is as large as the mean, since the time each vehicle spends between RSUs depends on its path. A longest gap of 7–14 s per handover remains in every configuration. Tracing it shows that the emulator reports the new association within a second, but the first packet reaches the new RSU's switch only 6–10 s later, when the controller logs the handover. The delay is the emulated station's scan and re-association across two channels, before any packet reaches the SDN layer, so no controller policy can remove it. It is a limitation of the wireless emulation rather than of the slicing design.

V. Conclusion

This paper presented a capacity-aware SDN orchestrator for V2X slicing and evaluated it through an 80-trial ablation on an emulated vehicular edge testbed. Under a broadband flood, any form of slicing kept critical delay near 25 ms with no loss, against several seconds without slicing. When critical load was heavy enough to oversubscribe the backhaul, metering alone was not sufficient (203 ms), whereas measurement-driven reallocation or strict priority each restored about 23 ms, and the full orchestrator gave the lowest delay of all configurations. Edge placement kept safety traffic near 1 ms.

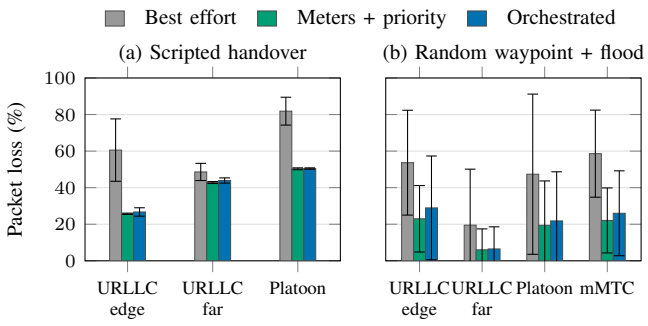


Fig. 6. Packet loss under mobility (mean of five repetitions, one standard deviation). (a) Scripted handover. (b) Random-waypoint mobility with the eMBB flood.

The main challenges lay below the controller: stale station entries and in-AP bridging both broke isolation silently until they were found and fixed, and the emulated Wi-Fi re-association still causes a 7–14 s gap at each handover, which inflates the losses measured under mobility. The loop also counts critical traffic before policing, so a misbehaving critical flow could shrink the broadband slice unfairly.

Future work will shorten re-association with single-channel deployment and faster scanning, then repeat the mobility study; steer safety flows to the nearest MEC after a handover; drive vehicles from a road-network simulator; scale the topology to more RSUs and vehicles; and base the loop on post-policing counters.

References

[1] 3GPP, "Enhancement of 3GPP support for V2X scenarios; stage 1 (release 16)," 3rd Generation Partnership Project, Technical Specification TS 22.186, 2019.

[2] M. H. C. Garcia, A. Molina-Galan, M. Boban, J. Gozalvez, B. Coll-Perales, T. Şahin, and A. Kousaridas, "A tutorial on 5G NR V2X communications," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 3, pp. 1972–2026, 2021.

[3] X. Foukas, G. Patounas, A. Elmokashfi, and M. K. Marina, "Network slicing in 5G: Survey and challenges," *IEEE Communications Magazine*, vol. 55, no. 5, pp. 94–100, 2017.

[4] I. Afolabi, T. Taleb, K. Samdanis, A. Ksentini, and H. Flinck, "Network slicing and softwarization: A survey on principles, enabling technologies, and solutions," *IEEE Communications Surveys & Tutorials*, vol. 20, no. 3, pp. 2429–2453, 2018.

[5] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "OpenFlow: Enabling innovation in campus networks," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69–74, 2008.

[6] D. Kreutz, F. M. V. Ramos, P. E. Veríssimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-defined networking: A comprehensive survey," *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14–76, 2015.

[7] C. Campolo, A. Molinaro, A. Iera, and F. Menichella, "5G network slicing for vehicle-to-everything services," *IEEE Wireless Communications*, vol. 24, no. 6, pp. 38–45, 2017.

[8] I. Ku, Y. Lu, M. Gerla, R. L. Gomes, F. Ongaro, and E. Cerqueira, "Towards software-defined VANET: Architecture and services," in *Proc. 13th Annual Mediterranean Ad Hoc Networking Workshop (MED-HOC-NET)*, 2014, pp. 103–110.

[9] C. Campolo, R. d. R. Fontes, A. Molinaro, C. E. Rothenberg, and A. Iera, "Slicing on the road: Enabling the automotive vertical through 5G network softwarization," *Sensors*, vol. 18, no. 12, p. 4435, 2018.

[10] T. d. V. Saraiva, C. A. V. Campos, R. d. R. Fontes, C. E. Rothenberg, S. Sorour, and S. Valaee, "An application-driven framework for intelligent transportation systems using 5G network slicing," *IEEE Transactions on Intelligent Transportation Systems*, vol. 22, no. 8, pp. 5247–5260, 2021.

[11] R. R. Fontes, S. Afzal, S. H. B. Brito, M. A. S. Santos, and C. E. Rothenberg, "Mininet-WiFi: Emulating software-defined wireless networks," in *Proc. 11th International Conference on Network and Service Management (CNSM)*, 2015, pp. 384–389.

[12] K. Nichols, S. Blake, F. Baker, and D. L. Black, "Definition of the differentiated services field (DS field) in the IPv4 and IPv6 headers," Internet Engineering Task Force, RFC 2474, 1998.

[13] J. Gettys and K. Nichols, "Bufferbloat: Dark buffers in the Internet," *Communications of the ACM*, vol. 55, no. 1, pp. 57–65, 2012.